SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE:CSA1723**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 30%

Duration : 45 Minutes

QUESTIONS: 2 **Total Marks:20**

Annexure A

Descriptive Written Test

Code of Conduct:

*I **SHREE AISHWARYA .R(192521475)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Shree Aishwarya .R

Signature of the student:

Question 1: Intelligent Agents in a Smart Home Automation System

1. Introduction

A smart home automation system uses intelligent agents to control lighting, temperature, and security by analyzing environmental conditions and user behavior. These agents receive information through sensors, make decisions, and perform actions through connected devices. The primary objective is to provide comfort, improve energy efficiency, and ensure security.

2. Types of Intelligent Agents

2.1 Simple Reflex Agent

A simple reflex agent selects actions based on the current percept using predefined condition–action rules. It does not consider past experiences or maintain an internal representation of the environment.

Application:

- Switches on lights when motion is detected.
- Activates cooling when the temperature exceeds a specified threshold.
- Triggers an alarm when a defined security condition occurs.

Advantages: Simple design, fast response, and low computational requirements.

Limitations: Cannot learn user preferences or effectively handle situations that require memory and contextual reasoning.

2.2 Model-Based Agent

A model-based agent maintains an internal state representing relevant information about the environment. It combines current sensor readings with previous observations to make informed decisions.

Application:

- Tracks room occupancy using motion sensor history.
- Considers indoor temperature and previous air-conditioning activity.
- Distinguishes between temporary movement and prolonged absence.

Advantages: Handles partially observable environments and supports context-aware decisions.

Limitations: Requires additional memory and an accurate internal model of the environment.

2.3 Goal-Based Agent

A goal-based agent selects actions according to specific objectives that it aims to achieve. It evaluates possible actions and chooses those that move the system toward its goals.

Application:

- Maintains a comfortable indoor temperature.
- Ensures doors and windows are secured when occupants leave.
- Turns off unnecessary appliances when the house is unoccupied.

Advantages: Supports planning, flexible decision-making, and achievement of multiple operational objectives.

Limitations: Planning can require significant computation, particularly when several possible actions and conditions must be considered.

2.4 Utility-Based Agent

A utility-based agent evaluates possible outcomes using a utility function and selects the action that provides the greatest overall benefit. It can balance competing objectives such as comfort, electricity consumption, and security.

Application:

- Adjusts air-conditioning to balance comfort and energy consumption.
- Uses occupancy and daylight information to optimize lighting.
- Prioritizes security when suspicious activity is detected, even if another action would consume less energy.

Advantages: Supports trade-offs between competing objectives and enables more flexible optimization.

Limitations: Designing suitable utility functions and assigning appropriate priorities can be complex.

3. Comparison of Intelligent Agents

Agent type	Decision basis	Main advantage	Main limitation
Simple reflex	Current percept and rules	Fast response	No internal memory
Model-based	Current input and internal state	Context-aware decisions	Requires an environment model
Goal-based	Desired objectives	Planning and goal achievement	Computationally demanding
Utility-based	Utility of possible outcomes	Balances competing objectives	Requires a suitable utility functi

4. Most Effective Approach

A **hybrid intelligent agent combining model-based, goal-based, and utility-based reasoning, supported by simple reflex rules**, is the most effective solution for a smart home automation system.

The model-based component maintains information about occupancy, temperature, and previous user activity. The goal-based component identifies objectives such as maintaining comfort and securing the home. The utility-based component balances comfort, energy efficiency, and security when selecting actions. Simple reflex rules provide immediate responses to urgent events.

For example, when a room becomes unoccupied, the system can use occupancy history to determine whether the absence is temporary, switch off unnecessary lights, and adjust the air-conditioning. However, if a security threat is detected, the system can prioritize protective action.

5. Conclusion

A hybrid intelligent agent provides a balanced solution for smart home automation. By combining rapid rule-based responses, environmental awareness, goal-oriented planning, and utility-based optimization, it can improve comfort, reduce unnecessary energy consumption, and strengthen security.

Question 2: Uninformed Search Strategies for Robot Maze Navigation

1. Introduction

A robot navigating an unknown maze must identify a path from its starting position to an exit without having prior knowledge of the maze layout. It can use sensors to detect walls and open passages, while uninformed search algorithms systematically explore available states without relying on heuristic estimates of the distance to the goal.

Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) are two important uninformed search strategies that can be applied to this problem.

2. Uniform Cost Search (UCS)

Uniform Cost Search expands the node with the lowest cumulative path cost, (g(n)). It uses a priority queue to select the least-cost path discovered so far.

Working procedure:

1. Initialize the starting position with a path cost of zero.
2. Insert the starting node into a priority queue.
3. Remove the node with the lowest cumulative cost.
4. Check whether the node represents the exit.
5. If it is not the goal, explore its accessible neighboring cells and calculate their cumulative costs.
6. Update the priority queue whenever a lower-cost path is found.
7. Continue until the exit is reached or no reachable nodes remain.

Advantages:

- Finds an optimal solution when step costs are non-negative and the required search conditions are satisfied.
- Handles paths with different movement costs.
- Is complete under standard conditions, including a finite branching factor and a positive minimum step cost.

Disadvantages:

- Can consume considerable memory because it stores frontier nodes and their associated costs.
- May explore many low-cost paths before reaching the goal.
- Can be computationally expensive in large mazes.

3. Iterative Deepening Search (IDS)

Iterative Deepening Search repeatedly performs depth-limited depth-first search, increasing the depth limit after each unsuccessful iteration. It combines the low memory requirement of depth-first search with the completeness of breadth-first exploration.

Working procedure:

1. Begin with a depth limit of zero.
2. Perform depth-first search up to the current limit.
3. If the exit is not found, increase the depth limit by one.
4. Repeat the search from the starting position.
5. Stop when the exit is discovered or the search space is exhausted.

Advantages:

- Requires relatively little memory.
- Is complete when the branching factor is finite.
- Finds a shallowest solution when every move has the same cost.
- Is suitable for large search spaces where memory is limited.

Disadvantages:

- Repeatedly explores upper-level nodes.
- Can perform unnecessary work when the solution is deep.
- Does not guarantee the least-cost path when movement costs differ.

4. Time and Space Complexity

Let (b) represent the branching factor, (d) the depth of the optimal solution, and (C^*) the optimal path cost. For UCS, assume every step cost is at least (\epsilon>0).

Parameter	Uniform Cost Search	Iterative Deepeni Search
Time complexity	(O(b^{1+\lfloor C^*/\epsilon\rfloor})) in the standard worst-case bound	(O(b^d))
Space complexity	Exponential; commonly bounded by the number of generated frontier and explored nodes	(O(bd))
Completeness	Yes, under standard positive-cost conditions	Yes, for finite bra factor
Optimality	Yes, for non-negative costs under standard UCS assumptions	Yes, for equal step

UCS is appropriate when movement costs vary and finding the least-cost path is essential. IDS is preferable when memory is limited and all movements have equal cost. Both algorithms assume that the robot can identify available neighboring states; a robot in a truly unknown maze must discover these states through sensing and exploration.

5. Relationship to Intelligent Agents

5.1 Simple Reflex Agent

A simple reflex agent responds to current sensor inputs using predefined rules, such as turning left when the forward path is blocked. It can handle immediate movement decisions but may repeatedly visit the same locations because it does not maintain a history of explored states.

5.2 Model-Based Agent

A model-based agent maintains an internal representation of discovered passages, blocked cells, and previously visited locations. As the robot explores, it updates this representation and uses it to avoid unnecessary repetition.

5.3 Goal-Based Agent

A goal-based agent selects actions that help it reach the exit. It uses search strategies to evaluate possible paths and determine which sequence of movements can achieve the goal.

5.4 Utility-Based Agent

A utility-based agent evaluates possible routes according to factors such as travel cost, time, battery consumption, and risk. It can select a route that offers the best overall outcome rather than considering distance alone.

6. Recommended Agent and Search Strategy

For a robot navigating an unknown maze, the most effective general approach is a **model-based, goal-based agent using Iterative Deepening Search for equal-cost movements**.

The model-based component records discovered passages and visited locations. The goal-based component establishes reaching the exit as the primary objective. IDS explores progressively deeper paths while maintaining relatively low memory requirements, making it suitable for environments where the maze size is large or initially unknown.

If the maze contains movements with different costs, such as energy-intensive terrain or risky passages, **Uniform Cost Search is the better choice** because it prioritizes the lowest cumulative path cost. A utility-based component can further evaluate energy consumption and safety when these factors are important.

The robot should also incorporate real-time sensing so that its internal maze representation can be updated when new passages or obstacles are discovered. Search decisions must therefore be integrated with environmental observation rather than relying on an assumed complete map.

7. Conclusion

Uniform Cost Search and Iterative Deepening Search provide complementary solutions to maze navigation. UCS is suitable for finding the least-cost path when movement costs vary, whereas IDS offers memory-efficient search and finds a shallowest solution when all moves have equal cost. Combining these algorithms with model-based and goal-based agent design enables a robot to explore an unknown maze systematically, adapt to newly discovered information, and reach its exit effectively

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**Max Marks**
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand